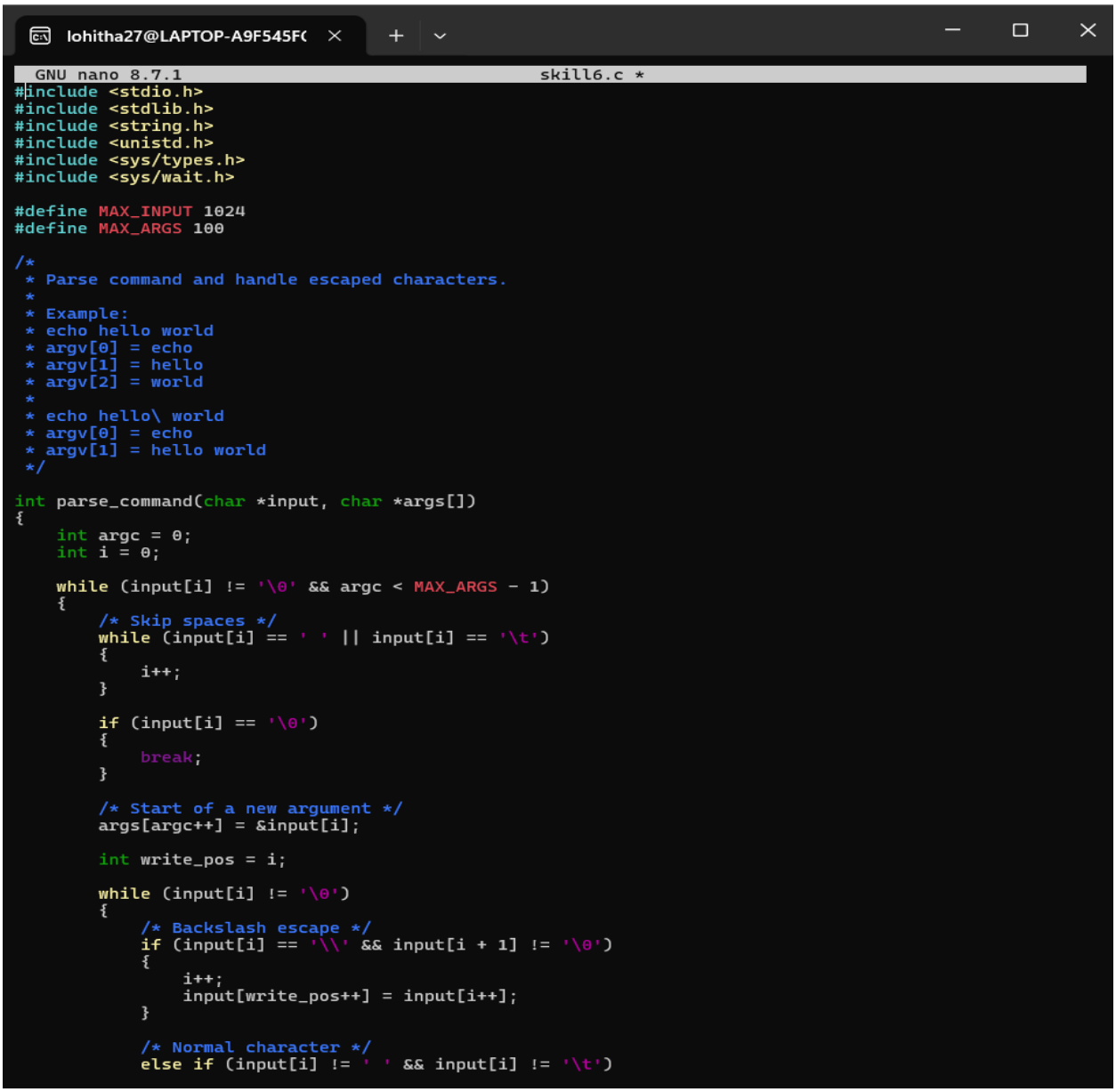
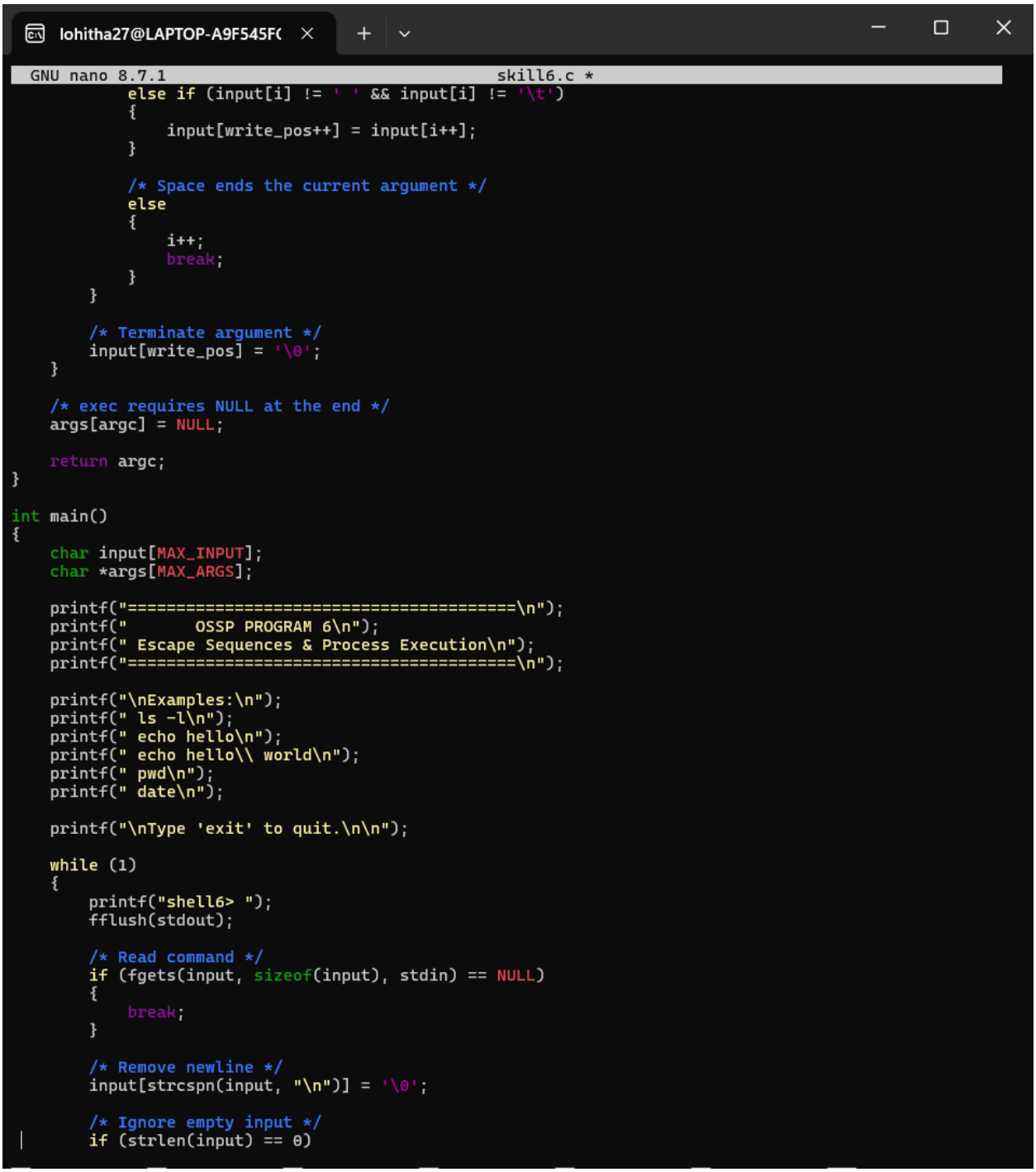
OS & SP

Name : S LOHITHA **Roll No :** 2520030109

SKILL – 06

1.Skill6.c




lohitha27@LAPTOP-A9F545F(
GNU nano 8.7.1 skill6.c *
else if (input[i] != ' ' && input[i] != '\t')
{
input[write_pos++] = input[i++];
}
/* Space ends the current argument */
else
{
i++;
break;
}
}
/* Terminate argument */
input[write_pos] = '\0';
}
/* exec requires NULL at the end */
args[argc] = NULL;
return argc;
}
int main()
{
char input[MAX_INPUT];
char *args[MAX_ARGS];
printf("==\n");
printf(" OSSP PROGRAM 6\n");
printf(" Escape Sequences & Process Execution\n");
printf("==\n");
printf("\nExamples:\n");
printf(" ls -l\n");
printf(" echo hello\n");
printf(" echo hello\\ world\n");
printf(" pwd\n");
printf(" date\n");
printf("\nType 'exit' to quit.\n\n");
while (1)
{
printf("shell6> ");
fflush(stdout);
/* Read command */
if (fgets(input, sizeof(input), stdin) == NULL)
{
break;
}
/* Remove newline */
input[strcspn(input, "\n")] = '\0';
/* Ignore empty input */
if (strlen(input) == 0)


lohitha27@LAPTOP-A9F545F(
GNU nano 8.7.1 skill6.c *
{
continue;
}
/* Exit */
if (strcmp(input, "exit") == 0)
{
break;
}
/* Parse command */
int argc = parse_command(input, args);
if (argc == 0)
{
continue;
}
/* Display arguments */
printf("\nParsed arguments:\n");
for (int i = 0; i < argc; i++)
{
printf(" argv[%d] = [%s]\n", i, args[i]);
}
/* Create child process */
pid_t pid = fork();
if (pid < 0)
{
perror("fork");
continue;
}
/* CHILD PROCESS */
if (pid == 0)
{
printf("\nChild process created.\n");
printf("Child PID: %d\n", getpid());
/* Execute command */
execvp(args[0], args);
/* Reached only if execvp fails */
perror("execvp");
exit(EXIT_FAILURE);
}
/* PARENT PROCESS */
else
{
printf("Parent PID: %d\n", getpid());
printf("Child PID: %d\n", pid);
printf("Parent waiting for child...\n");
int status;
^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location M-U Undo
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo
94
ENG
IN
09:12
13-10-2026

```
            int status;

            /* Wait for child to finish */
            waitpid(pid, &status, 0);

            /* Check child status */
            if (WIFEXITED(status))
            {
                printf(
                    "Child exited with status: %d\n",
                    WEXITSTATUS(status)
                );
            }
            else
            {
                printf("Child terminated abnormally.\n");
            }
        }

        printf("\n");
    }

    printf("Program 6 terminated.\n");
return 0;
}
```

OUTPUT :

```
parser> ^C
lohitha27@LAPTOP-A9F545FG:~/SKILL$ nano skill5.c
lohitha27@LAPTOP-A9F545FG:~/SKILL$ nano skill6.c
lohitha27@LAPTOP-A9F545FG:~/SKILL$ gcc skill6.c -o skill6
lohitha27@LAPTOP-A9F545FG:~/SKILL$ ./skill6
=====================================
       OSSP PROGRAM 6
 Escape Sequences & Process Execution
=====================================

Examples:
 ls -l
 echo hello
 echo hello\ world
 pwd
 date

Type 'exit' to quit.

shell6> valgrind --leak-check=full --show-leak-kinds=all ./program6

Parsed arguments:
 argv[0] = [valgrind]
 argv[1] = [--leak-check=full]
 argv[2] = [--show-leak-kinds=all]
 argv[3] = [./program6]
Parent PID: 1276
Child PID: 1279
Parent waiting for child...

Child process created.
Child PID: 1279
execvp: No such file or directory
Child exited with status: 1

shell6> echo hello

Parsed arguments:
 argv[0] = [echo]
 argv[1] = [hello]
Parent PID: 1276
Child PID: 1282
Parent waiting for child...

Child process created.
Child PID: 1282
hello
Child exited with status: 0

shell6>
```